

Resource Access Protocols in Real-Time Embedded Systems

Raihan Uddin

May 26, 2025

Contents

1	Introduction to Resource Sharing in Real-Time Systems	2
----------	--	----------

1 Introduction to Resource Sharing in Real-Time Systems

A new process in real-time system starts with assigning different resources. These resources may vary from data structures, variables, memory (both volatile and non-volatile), files, registers, I/O to processors and others- depending on the task. As stated by [1], "Starting a new process is a heavy job for the OS, which includes allocating memory, creating data structures, and coping code." In real-time systems, the process of resource sharing is crucial for ensuring that tasks can execute without unnecessary delays or conflicts and thus meeting their deadlines. As stated by [1] also, "Real-time embedded systems are often run in highly resource-constrained environments, which make the system design and performance optimization quite challenging."- thus implying the importance of sharing resources. The process needs to follow certain protocols. These protocols can be defined as "a set of rules that govern when and under what conditions each request for resource is granted and how tasks requiring resources are scheduled." [1]. Whereas it is also mentioned by [2] that, "Any concurrent operating system (OS) must offer protocols to control the access to resources shared (i.e., a piece of code, such as data structures, I/O devices, buffers, and so on) by competing tasks, thus ensuring mutual exclusion in their respective critical sections (a code segment where shared variables can be accessed)". To further emphasize the importance, a well-designed access control protocol is critical for preventing deadlocks from occurring [1]. Although, there is no protocol that can eliminate priority inversion, the goal of access control is to keep the blocking time of the higher priority tasks under control [1].

For better understanding of how these protocols manage resources allocation among different tasks, a situation can be considered where 3 tasks (J1, J2 and J3) are running in a real-time system. There, are m type of resources in a system, namely $R_1, R_2, R_3, \dots, R_m$ where each resource comprises η_i number of indistinguishable units of R_i , $i=1, 2, \dots, m$. When a task requests ρ_i units of a resource R_i , request is sent to kernel to lock them and can be denoted as $L(R_i, \rho_i)$. When the resource is longer needed by the task, it is unlocked and can be denoted by $U(R_i, \rho_i)$. As plentiful resources have no effect on scheduling plentiful resources have no effect on scheduling [?], we can avoid considering them here. Also to be considered here- only the serially reusable resources, which are "one that can be used safely by one task at a time and is not depleted by that use" [1]. The examples of these resources are devices, files, databases, and semaphores [1]. The access to resources by

every task will be controlled by locking mechanisms.

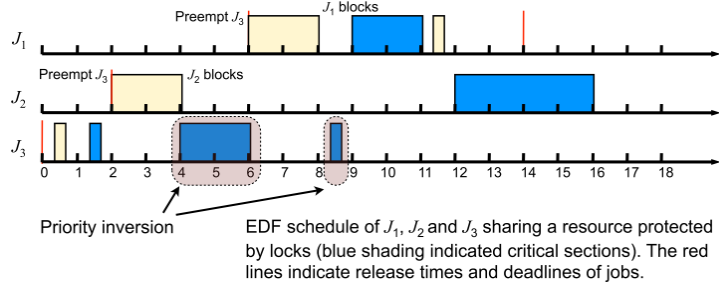


Figure 1: EDF scheduling of tasks while resources are protected by locks [3]

References

- [1] Jiacun Wang, *Real-Time Embedded Systems*. Quantitative Software Engineering Series, 111 River Street, Hoboken, NJ 07030, USA: John Wiley & Sons, Inc., 2017.
- [2] L. M. dos Santos, G. Gracioli, T. Kloda, and M. Caccamo, “Supporting single and multi-core resource access protocols on object-oriented rtoses,” vol. 27, pp. 31–50, 2023.
- [3] “Resource access control in real-time systems: Advanced operating systems (m).”